```
// Cloudflare Worker: speech-to-text for
Caption Studio (uses Gemini API audio
understanding; key stays secret).
// Create a Worker named "smart-caption".
Variables: GEMINI_API_KEY (Secret,
required), GEMINI_MODEL (optional, default
gemini-3.7-flash), ALLOWED_ORIGIN (your
site)
export default {
  async fetch(req, env) {
    const cors = { "Access-Control-Allow-
Origin": env.ALLOWED_ORIGIN || "*",
"Access-Control-Allow-Methods": "POST,
OPTIONS", "Access-Control-Allow-Headers":
"Content-Type" };
    const J = (o, s = 200) =>
Response.json(o, { status: s, headers: cors
});
    if (req.method === "OPTIONS") return
new Response(null, { headers: cors });
    if (req.method !== "POST") return new
Response("Caption worker is running", {
headers: cors });
    try {
      const { audio, lang } = await
req.json();
      if (!audio) return J({ error: "No
audio" }, 400);
      const script = lang === "हिन्दी" ?
"Hindi in Devanagari script" : lang ===
```

```
"English" ? "English" : "Hinglish (Hindi
spoken words written in English/Roman
letters, natural spoken style)";
      const prompt = `Transcribe EXACTLY
what is spoken in this audio, word for
word, in the order spoken. Do NOT add,
summarize, improve or invent any words.
Write the text as ${script}. Split into
short caption lines of at most 6 words.
Return ONLY a JSON array like [{"start":
0.0, "text": "..."}] where start is the
time in seconds (number) when that line
begins. If there is no speech return [].`;
      const model = env.GEMINI_MODEL ||
"gemini-3.7-flash";
      const r = await
fetch(`https://generativelanguage.googleapi
s.com/v1beta/models/${model}:generateConten
t`, {
        method: "POST",
        headers: { "x-goog-api-key":
env.GEMINI_API_KEY, "Content-Type":
"application/json" },
        body: JSON.stringify({ contents: [{
parts: [{ text: prompt }, { inline_data: {
mime_type: "audio/wav", data: audio } }]
}], generationConfig: { responseMimeType:
"application/json", temperature: 0 } }),
      });
      const d = await r.json();
      if (!r.ok) return J({ error:
d.error?.message || "Gemini error" },
r.status);
      const t = (d.candidates?.
```

```
[0]?.content?.parts || []).map((p) =>
p.text || "").join("");
      const m = t.match(/\[[\s\S]*\]/);
      return J({ lines: m ?
JSON.parse(m[0]) : [] });
    } catch (e) { return J({ error: "Server
error" }, 500); }
  },
};
```